

Titanic - Machine Learning from Disaster Write Up

Sophie Claybrook

August 2026

1 Introduction

This project shows the process and the findings of a machine learning workflow to Kaggle's "Titanic - Machine Learning from Disaster", an introductory classification competition.

The aim of this report is to predict whether each passenger survived based on features such as age, sex, class and fare. The dataset is split into 891 training passengers (known outcomes) and 418 test passengers (unknown).

I have completed this project to help build hands-on machine learning experience beyond the theory covered in my degree.

This report will cover the data, methodology, model comparison, tuning and results.

2 The Data

Within the dataset there are 891 training rows, 418 test rows, described by 11 original features. The key features are Pclass (ticket class, 1st, 2nd or 3rd), Sex, Age, SibSp (siblings/spouses aboard), Parch (parents/children aboard), Ticket number, Fare, Cabin and Embarked (port of boarding: Southampton, Cherbourg, or Queenstown). The target variable, Survived, is binary (1 = survived, 0 = died).

For many columns, several values were missing. For example Age was missing for a large amount of the passengers, embarked was missing data for two passengers and cabin had little record, making them unsuitable to use.

One interesting fact I discovered in the data was that 74.20% of women survived, compared to 18.89% of men — a stark difference, and the survival rate was much higher for 1st class than 3rd class. This is consistent with the "women and children first" evacuation priority and the reality that first-class cabins were closer to lifeboat access [2].

3 Methodology

3.1 Data Cleaning

The missing values in Age were filled with the median age across the dataset, which ensured the distortion of outliers was avoided (if mean had been used). Any missing embarked values were filled with the value of the mode, the most common boarding point. The feature, cabin, was completely dropped due to the high proportion of missing data, which made imputation unreliable.

3.2 Future Engineering

Additionally, a new feature title was implemented. Title was extracted from passenger's name, since title can show a person's age, sex and social status. Rare titles were identified and separated to avoid the model treating one-off values as patterns. FamilySize was an added feature, created by the addition of SibSp and Parch, with the addition of the passenger. This captured the size of the group travelling and their survival odds.

3.3 Encoding

Sex, Embarked and title are considered categorical variables, therefore they were converted into binary outputs (because machine learning models require numerical inputs). The training and test sets were then aligned, to ensure that all columns matched and any additional columns were removed.

3.4 Train/Validation Split

Following this, the training data was split 80/20 into a training subset and a test subset. This allowed the model's performance to be based on data the model had not seen during training.

3.5 Models Tested

Three classification algorithms were compared: Logistic Regression (a linear model which estimates the probability of survival), a Decision Tree (a single rule-based tree which works by splitting data on feature thresholds), and a Random Forest (a collection of decision trees whose predictions are averaged, reducing overfitting compared to a single tree).

3.6 Hyperparameter tuning

To search over combinations of Random Forest Settings, GridSearchCV was used: number of trees (n_estimators: 100/200/300), maximum tree depth (max_depth: unlimited/5/10), and minimum samples required to split a node (min_samples_split: 2/5/10) — using 5-fold cross-validation to estimate performance for all the combinations instead of relying on one train/test split.

4 Results

Model	Validation Accuracy
Logistic Regression	79.33%
Decision Tree	76.54%
Random Forest	83.24%

Table 1: Validation accuracy of the classification models

Among all the models, Random Forest showed the best performance. This was expected as combining many trees typically generalises better than a single decision tree, which faces the problem of over-fitting data [1].

Hyperparameter tuning via GridSearchCV identified n_estimators=300, max_depth=5, min_samples_split=2 as the best-performing combination, achieving a cross-validation accuracy of 83.28%. The performance of this model was slightly better than the untuned version. However, when evaluated on the held-out validation set, the tuned model achieved 81.56%, below the untuned Random Forest's 83.24%. This reflects the small size of the dataset. The original untuned Random Forest was selected for the actual submission achieving a submission score of 74.

5 Conclusion

Concluding this project, I can reflect the practical experience I have gained across the full applied machine learning method: cleaning real-world data with missing values, engineering meaningful features from raw fields, comparing multiple model types on a level footing, and using systematic hyper-parameter search to attempt to improve performance.

A key idea which I have gained is that hyper-parameter tuning does not guarantee improvement on every specific evaluation, An important lesson about the difference between cross-validation performance and performance on any single held-out sample, especially when the dataset is small.

To further extend this project I could include more feature engineering, testing additional model types, and exploring ensembling across the three tested models.

References

- [1] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [2] British Wreck Commissioner’s Inquiry. Testimony of george w. beauchamp, fireman, ss titanic. <https://www.titanicinquiry.org/BOTInq/BOTInq03Beauchamp01.php>, 1912. British Inquiry into the Loss of the Titanic.